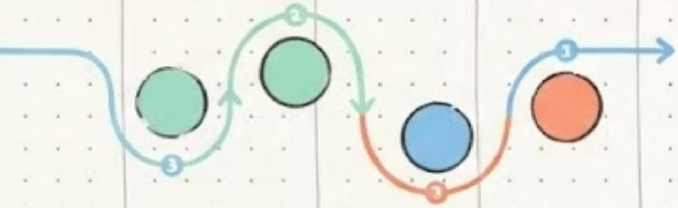


Gestão do Ciclo de Vida da Aplicação

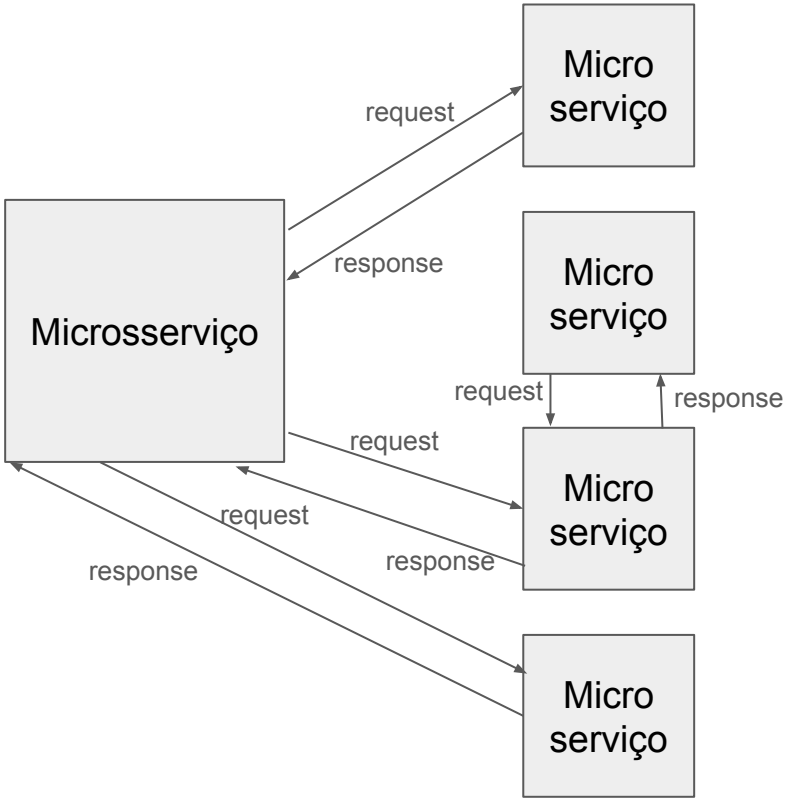
Arquitetura de sistemas (orientado a Mensagens e Pub/Sub)

Professor Romário da Silva Borges



Arquitetura Orientada a Mensagens

Modelo de comunicação até agora: request-response (REST, gRPC, GraphQL, etc)



Problema?

Arquitetura orientada a Mensagens

- Clientes não se comunicam diretamente com servidores
- Mas com um intermediário: **fila de mensagens** (ou broker de mensagens)



Exemplo: RabbitMQ

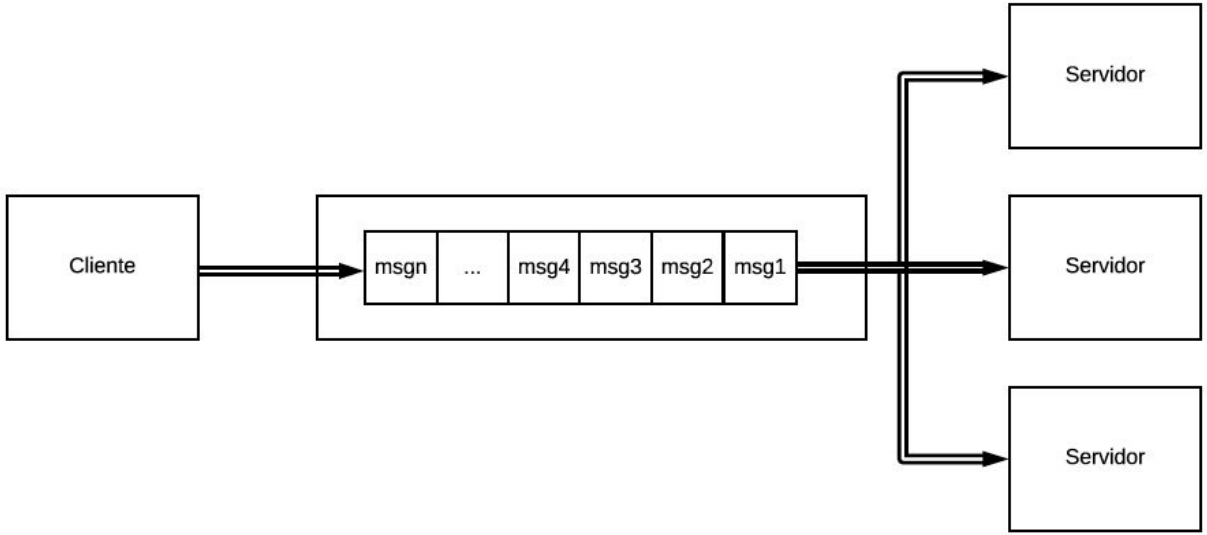
Vantagem #1: Tolerância a Falhas

- Não existe mais mensagem: "servidor fora do ar"
- Assumindo que a fila de mensagens roda em um servidor bastante robusto e confiável



Vantagem #2: Escalabilidade

- Mais fácil acrescentar novos servidores



Comunicação Assíncrona

- Comunicação entre clientes e servidores é **assíncrona**
- Cria um acoplamento fraco entre clientes e servidores

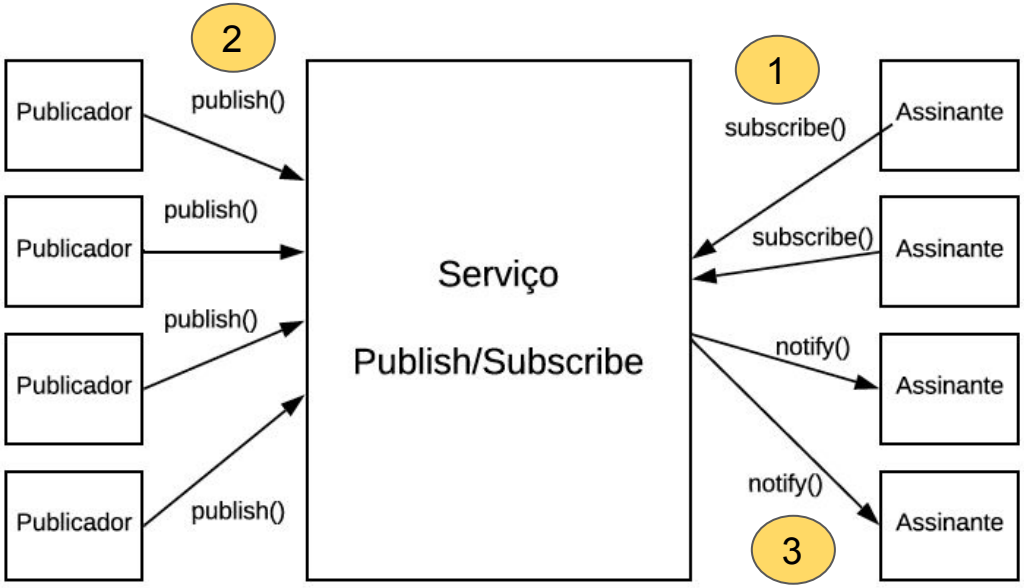
Arquitetura Publish/Subscribe

Publish/Subscribe

- "Aperfeiçoamento" de fila de mensagens
- Mensagens são chamadas de **eventos**

Publish/Subscribe

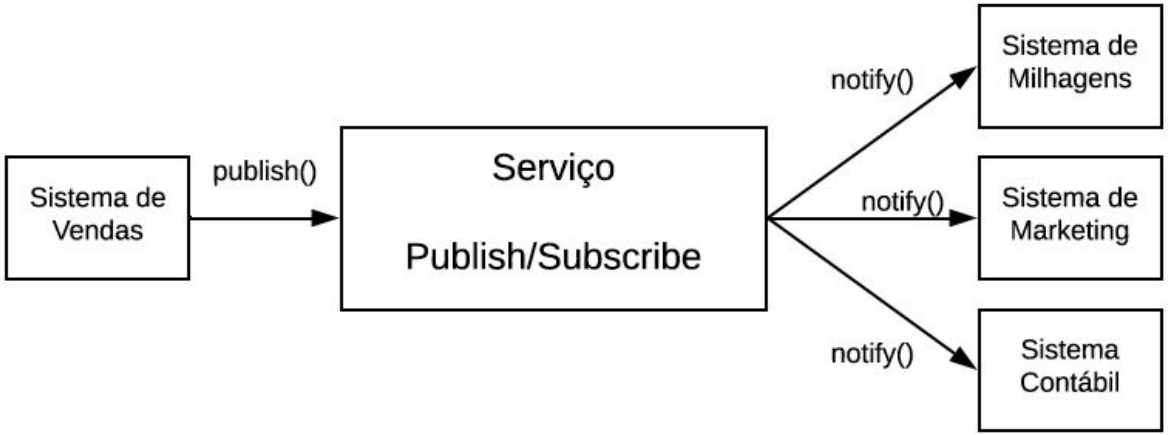
- Sistemas podem: (1) assinar eventos; (2) publicar eventos; (3) serem notificados sobre a ocorrência de eventos



Exemplo: Kafka

Exemplo: Sistema de Companhia Aérea

- Evento: venda de passagem (com dados da venda)



Comunicação em grupo: um sistema publica eventos, *n* sistemas assinam e são notificados da publicação

Roteiro prático sobre Publish/Subscribe

https://github.com/aserg-ufmg/pub-sub-store

Outros Padrões Arquiteturais

Outros Padrões Arquiteturais

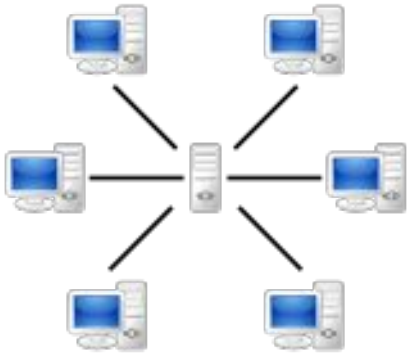
(1) Pipes e Filtros

- Programas são chamados de **filtros** e se comunicam por meio de **pipes** (que agem como buffers)
- Arquitetura bastante flexível. Usada por comandos Unix.
- Exemplo: `ls | grep csv | sort`



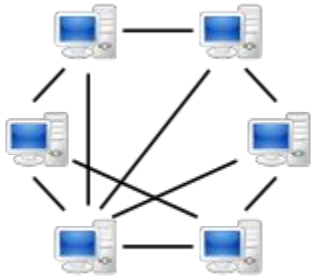
(2) Cliente/Servidor

- Muito comum em serviços básicos de redes
- Exemplos:
  - Serviço de impressão
  - Serviço de arquivos
  - Serviço Web



(3) Peer-to-Peer

- Todo nodo pode ser cliente e/ou servidor
- Isto é, pode ser consumidor e/ou provedor de recursos
- Exemplo: sistemas para compartilhamento de arquivos (usando BitTorrent); Blockchain



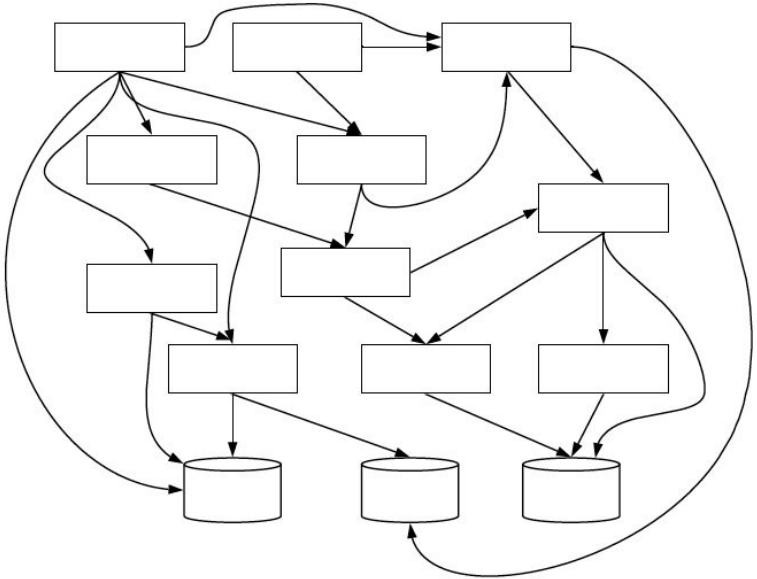
Anti-padrões Arquiteturais

Anti-padrão

- Modelo que não deve ser seguido
- Revela um sistema com sérios problemas arquiteturais

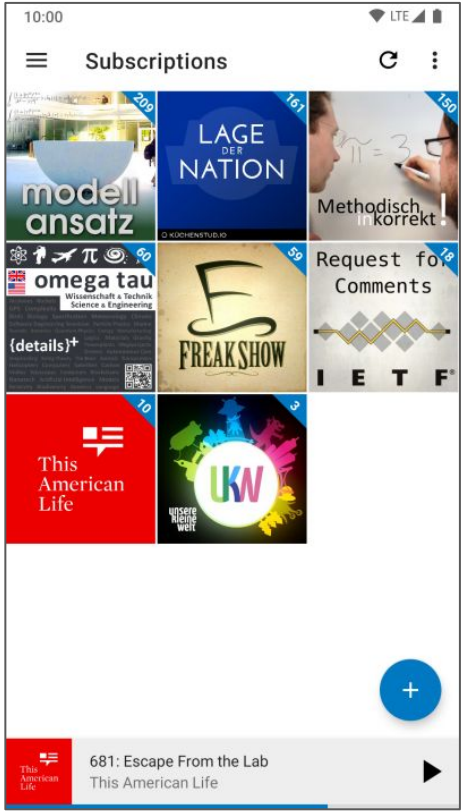
Big Ball of Mud

- Um módulo pode usar praticamente qualquer outro módulo do sistema; ou seja, sistema é uma "bagunça"



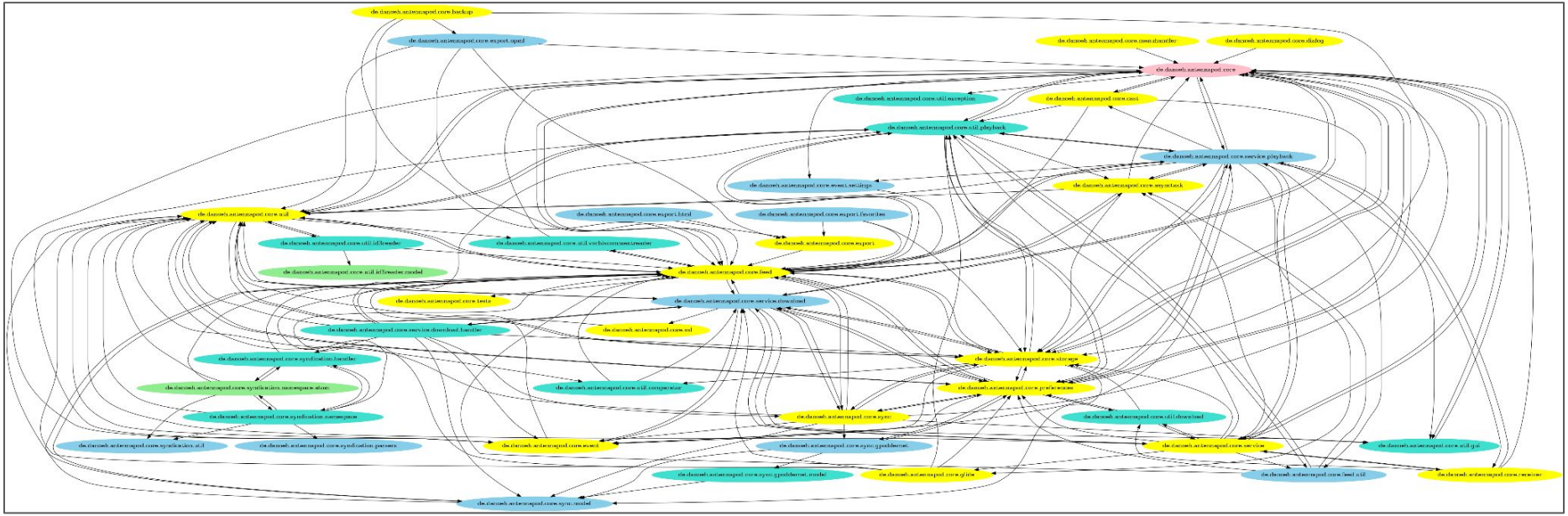
Exemplo de Remodularização

- AntennaPod: sistema open-source para tocar podcasts
- Android e Java



https://antennapod.org

Novembro, 2020 - Big Ball of Mud (com diversas dependências circulares)

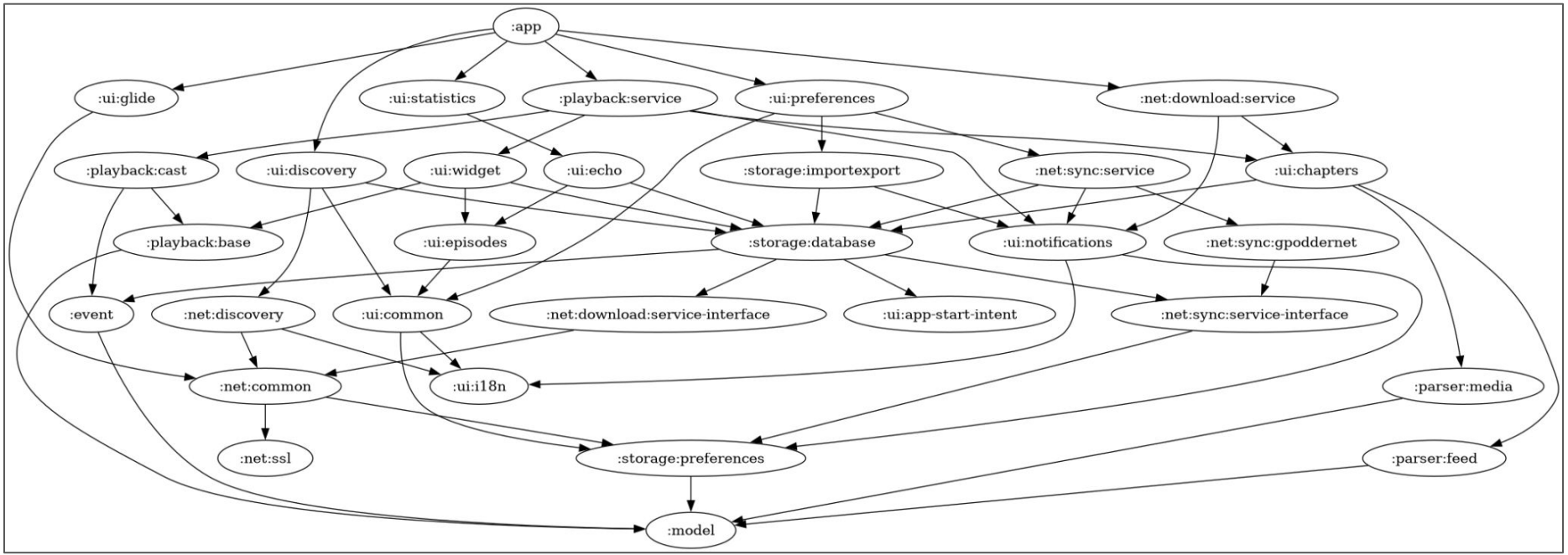


https://antennapod.org/blog/2024/05/modernizing-the-code-structure

Comentários dos desenvolvedores

- "To test the database, for example, one normally wouldn’t have to launch the full app. However, because the database basically depended on everything else, most of our tests required starting up a full Android device."

Maio, 2024



https://antennapod.org/blog/2024/05/modernizing-the-code-structure

Exercicios

1. Qual a provável arquitetura dos seguintes sistemas? Justifique brevemente sua resposta.

   (a) Microsoft Excel (versão para desktop)

   (b) Nubank App (mobile)

   (c) Twitter/X Web (front-end)

   (d) Google Slides (front-end)

   (e) Twitter/X (backend)

   (f) Moodle

2. Responda sobre microsserviços:

(a) Por que uma arquitetura baseada em microsserviços oferece flexibilidade para os times colocarem seu código em produção de forma independente?

(b) Por que isso é menos viável em uma arquitetura monolítica? Ou seja, por que não é recomendável que um time, ao fazer uma modificação em um monolito, coloque-a em produção imediatamente?

(c) Por que microsserviços não devem compartilhar o mesmo BD?

3. Suponha uma empresa de streaming que deseja implementar um sistema detector de problemas de qualidade em seus vídeos (por exemplo, problemas nas legendas, no áudio, congelamento de imagens, etc). Os vídeos estão todos armazenados em um sistema de storage, isto é, em memória secundária. A empresa está avaliando duas arquiteturas:

Arquitetura #1: cada detector é um microsserviço, que recebe o nome do vídeo como parâmetro, carrega o mesmo do storage e executa um algoritmo de detecção de problemas de qualidade nesse vídeo.

Arquitetura #2: os detectores são módulos de um monolito. Então, o vídeo é carregado uma única vez do storage para a memória principal e compartilhado por todos os detectores de qualidade.

Supondo que a empresa de streaming tem milhões de clientes (ou seja, é uma empresa do porte da Amazon Prime Video) qual arquitetura você considera mais escalável? Justifique.

Este exercício é baseado em um caso real documentado em um blog da Amazon Prime Video. No contexto do exercício, escalabilidade significa manter os custos sob controle mesmo quando número de vídeos analisadores cresce bastante.





Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

